F1 Overtake Prediction Simulation - Project Report

Chathura Dharmasiri BSc (Hons) in Computer Science University of Vavuniya

1. Executive Summary

The **F1 battery SoC calculation and Prediction Sim.** is a real-time data engineering and machine learning pipeline designed to predict Formula 1 overtakes as they happen. By tapping directly into Formula 1's live telemetry WebSockets, the system streams high-frequency data (Speed, Throttle, Brake, RPM, etc.), engineers advanced features like Battery State of Charge (SOC), predicts overtakes using an XGBoost model, and visualizes the results on a live Streamlit dashboard.

2. System Architecture & Data Flow

The project is built on a modern, low-latency stack designed for real-time streaming and time-series analysis.

Component Breakdown

1 Data Ingestion (`livef1_client.py` / `fastf1_recorder.py`)

- Connects to F1's undocumented SignalR WebSockets.
- Intercepts and decompresses high-frequency streams (`CarData.z` for telemetry, `Position.z` for GPS, and `TimingData` for gaps and sector times).
- *Requirement:* A valid F1 TV Access subscription is used to authenticate the connection and unlock the premium telemetry channels.

2 Feature Engineering (`src/common/soc_calculator.py`)

- Raw telemetry isn't enough to predict an overtake; we need to know how much electrical energy a driver has left.
- The custom **SOC Calculator** estimates the battery's State of Charge by evaluating throttle and braking application over time (e.g., harvesting under braking, deploying under full throttle).

3 Machine Learning Model (`src/ml_model/predict.py`)

- The pipeline passes live Speed, Throttle, Brake, Gap, and SOC into an XGBoost classification model.
- The model outputs a binary prediction indicating whether an overtake is imminent on the current lap.

4 Time-Series Database (schema.cql)

- Data is persisted at high frequency into a locally-hosted **ScyllaDB/Cassandra** cluster *(Redis will be used in future)*.
- The keyspace `f1_live.live_telemetry` is clustered by timestamp descending to ensure the dashboard can read the absolute latest data instantly.

5 Real-Time Dashboard (src/dashboard/app.py)

- A highly-styled, responsive user interface built with **Streamlit** and Plotly *(Streamlit for now)*.
- **Timing Tower:** Shows live driver positions, gaps, tire compounds, and lap counts.
- **Driver Detail:** Interactive gauges for Speed, Throttle, Brake, and RPM.
- **ML Insights:** Displays the real-time "Overtake Likely" prediction alongside the calculated Battery SOC.

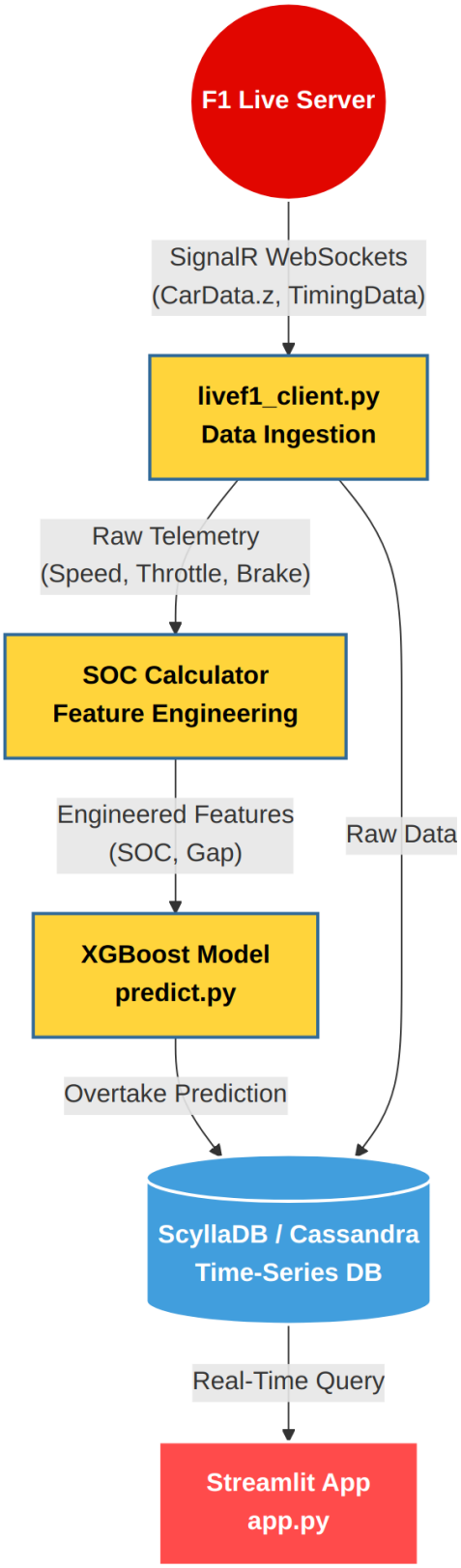

Figure 1 - End-to-end data flow from F1 Live Server to the Streamlit dashboard.

Authentication Requirement

F1 enforces strict bandwidth controls on live telemetry. While historical data can be analyzed for free, live real-time telemetry (CarData.z / Position.z) requires the client to authenticate with a valid F1 TV Access subscription. Unauthenticated (Guest) clients are heavily throttled and only receive standard TimingData.

3. Data Capture Requirements

Example Captured Record

When successfully authenticated, the pipeline captures and processes data in the following structure: *(For basic model)*

```
{
  "driver": "NOR",
  "timestamp": "2026-08-21T15:22:26.837Z",
  "speed": 315.2,
  "throttle": 100.0,
  "brake": 0.0,
  "rpm": 11500,
  "gap_to_ahead": 0.450,
  "estimated_soc": 68.4,
  "overtake_prediction": 1
}
```

4. Current Status & Next Steps

Item	Status
Infrastructure — ScyllaDB schema and local Docker connectivity	☑ Complete
Dashboard — Streamlit UI built, themed (F1 dark mode), gauges & speed trap comparisons	☑ Complete
Data Ingestion — Scripts ready to intercept SignalR streams	☑ Complete
Action Item — Ensure active F1 TV Access subscription is passed into fastf1/livef1 client to unlock CarData.z during next live session	In Progress